

邏輯系統作業一：自動 K-map 求解器實作報告

學號：E24146107
課程：邏輯系統 (Logic System)

1. AI 互動與審核紀錄 (AI Interaction & Review Evidence)

使用工具與模型 (AI Tools & Model Used)： Gemini CLI Agent (Model: Gemini 3 Pro / Flash)

本作業採用 AI 協助開發，並由本人（使用者）擔任 **審核者 (Auditor)**，針對 AI 產生的邏輯進行精確校對與修正。以下為各階段的審核重點：

階段一：座標映射與 Gray Code 順序校對

[過程] AI 初步產生的座標映射邏輯與本人習慣的橫縱軸分配相反。**[審核與修正]** 本人指出橫軸應由 CD 決定，縱軸由 AB 決定。要求 AI 修正座標映射邏輯，以符合以下 K-map 實體分佈：

審核通過之 K-map 索引分佈圖：

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

階段二：畫圈邏輯 (Sub-cube Searching) 驗證

[過程] 驗證程式是否能正確處理「捲曲相鄰 (Wrapping)」的情況。**[審核與校對]** 本人對「四個角落 (0, 2, 8, 10)」進行人工邏輯推演。

- 0 (0000), 2 (0010), 8 (1000), 10 (1010)
- 固定位元為 B=0, D=0。
- [結論]** 確認邏輯項為 $B'D$ 。AI 的位元遮罩演算法成功通過此項測試。

階段三：EPI 篩選與 Minimum SOP 邏輯審核

[過程] 審核 Prime Implicant (PI) 如何被過濾成 Essential Prime Implicant (EPI)。**[審核與校對]** 給定範例 Minterms {1, 3, 5, 7, 13, 15}：

- PI $X=\{1,3,5,7\}$, $Y=\{5,13\}$, $Z=\{13,15\}$ 。
- 本人判定 Minterm 1 僅被 X 覆蓋，Minterm 15 僅被 Z 覆蓋。
- [結論]** X, Z 為 EPI，Y 為冗餘項。最終答案應為 $X + Z$ 。AI 實作之貪婪演算法結果與本人審核結果一致。

2. 演算法說明 (Program Explanation)

本程式完全避開 Quine-McCluskey 演算法，改用模擬圖形化 K-map 的 **Sub-cube 搜尋法**：

- 矩陣化**：依據 Gray Code 順序建立 2D char **kmap。
- 暴力遍歷搜尋**：在 2^4 空間內，窮舉所有大小為 16, 8, 4, 2, 1 的潛在方塊 (Sub-cubes)。
- PI 提取**：若一方塊內全為 1 或 X，且不被更大的方塊包含，則定義為 Prime Implicant。
- EPI 與 SOP 求解**：
 - 首先找出所有 EPI (覆蓋了至少一個「未被其他 PI 覆蓋」之 minterm)。
 - 若 EPI 尚未完全覆蓋所有 minterms，則使用貪婪演算法挑選剩餘能覆蓋最多 minterms 的 PI。

3. 偵錯過程 (Debugging Process)

在開發過程中遇到嚴重的 **系統死結 (System Deadlock)**：

- 現象**：Git 殘留 index.lock 檔案，導致背景監控服務 sync.py 崩潰。
- 影響**：由於 gatekeeper.py 限制，當監控服務離線時，無法執行任何檔案修改。
- 診斷與修復**：
 - 進入 Physical Diagnosis Mode，透過 git status 發現 Stale Lock。
 - 手動移除鎖定檔案並重啟守護進程。
 - 更新 PREVENTION_REPOSITORY.md 增加規則 25、26，確保未來偵測到基礎設施死結時可自動執行自癒路徑。

4. 邏輯驗證與測試結果 (Logic Verification & Test Results)

本程式通過以下測試案例，結果皆與 PDF 範例一致：

測試情境	輸入 Minterms	輸入 Don't Cares	輸出 Minimum SOP	狀態
PDF Page 2	1, 3, 5, 7, 9	6, 12, 13	$a'd + c'd$	Pass
PDF Page 4	2, 5, 7, 11, 13, 14	3, 10, 15	$ac + b'c + bd$	Pass
邊界測試 (2-var)	1	3	b	Pass

5. 進階 C++ 技術心得 (Advanced C++ Reflection)

- Bitmasking 應用**：使用 Mask 與 Value 的位元運算來代表方塊，取代複雜的字串比對，大幅提升搜尋效率。
- Lookup Tables (LUT)**：建立 `valToIndex` 查表，將 Gray Code 邏輯與座標轉換分離，使程式碼更具可讀性。
- 動態二維陣列管理**：深入理解 `char **` 的配置與釋放，確保在處理不同變數數量 (2x2, 2x4, 4x4) 時的記憶體安全。
- STL 容器優化**：利用 `std::set` 自動處理 PI 的重複項，並結合 `std::sort` 進行字母序排列。
- 貪婪演算法實作**：在處理多重覆蓋問題時，實作了一個高效的 Greedy Set Cover 邏輯。